



جامعة ابن زهر
+08/0444 400 000
UNIVERSITÉ IBN ZOHR



Program: **Software Engineering**

PROJECT REPORT

Design and Development of a Distributed Emergency Alert & Response Management System

Prepared by:

AGOURRAM Sihame
ALLABOUCH Anass
BELHOUFERT Safae
TAIKOUK Nassima
AIT BENAYAD Hamza
OUBAHA Khadija
BOUTBIB Ayoub
NEKHLA Amine

Supervised by:

Pr. EL HABOUZ Youssef

Academic Year : 2025/2026

Table of contents

List of Figures	4
Part I: Theoretical study	6
Chapter 1: General Presentation:	7
Introduction:	7
Project Overview:.....	7
Domain Challenges:	7
Functional Requirements:	7
Justification for the Choice of Topic:.....	8
Conclusion:.....	8
Introduction:	9
Computer Science Concepts of the Project:.....	9
Technologies Used:	10
Conclusion:.....	11
Chapter 3: Requirements Analysis	12
Introduction:	12
Use Case Diagram:.....	12
Class Diagram:	14
Sequence Diagram:	16
Deployment Diagram:	21
Conclusion:.....	23
Part II: Practical Study	24
Introduction:	25
Conclusion:.....	26
Chapter 5: Testing and Validation	27
Introduction:	27
Home page:	27
Register page:	30
Login page:.....	31
Declare emergency page :	32

Status page :	33
My alerts page:	34
Responder page:	35
Submit-report page :	36
Dashboard alerts:	37
Dashboard response page:	38
Responder's profile page:	39
Admin's profile page.....	40
citizen's profile page :	41
Conclusion:	41
Conclusion:	42

List of Figures

Figure 1: python logo	10
Figure 2:django logo	10
Figure 3:PostgreSQL logo	10
Figure 4:PostgreSQL	10
Figure 5:GitHub logo	11
Figure 6:use case diagram	12
Figure 7:Class Diagram	14
Figure 8:sequence diagram (citizen).....	16
Figure 9:sequence diagram (Admin)	18
Figure 10:sequence diagram (responder)	19
Figure 11 Deployment Diagram.....	21
Figure 12:home page	29
Figure 13:register page	30
Figure 14:login page	31
Figure 15:declare emergency page	32
Figure 16:status page	33
Figure 17:my alerts page	34
Figure 18:responder page	35
Figure 19:submit-report page	36
Figure 20:dashboard alerts	37
Figure 21:Dashboard response page	38
Figure 22:responder's profile page.....	39
Figure 23:Admin's profile page	40
Figure 24:citizen's profile page.....	41

Introduction

Emergency management systems play a crucial role in ensuring public safety and protecting lives during critical situations such as accidents, fires, crimes, and medical emergencies. The speed and accuracy with which alerts are handled can significantly influence the outcome of these events. However, many existing alert systems still rely on centralized or manual coordination mechanisms, which can lead to delays, system failures, or communication breakdowns when multiple emergencies occur simultaneously.

To address these challenges, the integration of distributed system technologies has become an essential solution. Distributed architectures enable the sharing of responsibilities among different autonomous services, improve system scalability, and ensure continuous operation even in the presence of component failures. In the context of emergency response, a distributed approach allows each specialized service—police, fire department, and medical units—to manage alerts independently while still remaining connected to a unified routing system.

This project focuses on the **design and development of a Distributed Emergency Alert & Response System (DEARS)**. The proposed system allows citizens to report emergency situations through a web interface, while an intelligent dispatcher routes these alerts to the appropriate response service using Remote Procedure Calls (RPC). Each service then processes the alert and coordinates the nearest available response unit in real time.

The primary objective of this work is to provide a reliable and efficient digital platform capable of minimizing intervention delays and enhancing coordination between emergency services. By leveraging distributed technologies and modern web development tools, DEARS aims to contribute to the continuous improvement of public safety and crisis response capabilities.

Part I: Theoretical study

Chapter 1: General Presentation:

Introduction:

The DEARS (Distributed Emergency Alert and Response System) project is part of a technological drive to enhance public safety services. This first chapter aims to establish the project framework by presenting its global objectives, the major challenges associated with emergency management in a distributed environment, and the motivations behind choosing this critical and complex topic.

Project Overview:

DEARS is a distributed system that enables the submission and processing of emergency alerts through a web interface and specialized backend services.

Domain Challenges:

- Rapid processing of alerts
- Reliability and consistency of data
- System scalability
- Separation of responsibilities between services

Functional Requirements:

- Submission of an alert by a citizen
- Automatic routing to the appropriate emergency service
- Management of intervention units
- Storage and tracking of alerts

Justification for the Choice of Topic:

The choice of this topic is justified by the importance of distributed systems in managing critical services, especially in emergency situations. This project allows the application of fundamental concepts such as distributed architecture, data modeling, and inter-service communication. It also provides a practical framework to apply theoretical knowledge acquired during coursework. Finally, the reliable and secure management of data represents a key and relevant aspect of this subject.

Conclusion:

In summary, this chapter has defined the scope of the DEARS system and identified critical functional requirements such as processing speed and data reliability. With these foundations established, it is now essential to examine the theoretical concepts and technologies required to meet these high-availability demands.

Chapter 2: Theoretical Foundations

Introduction:

Designing a distributed system for alert management requires a deep understanding of communication mechanisms and data storage. This chapter outlines the technological pillars of DEARS, ranging from RPC and REST communication protocols to relational database management systems, while justifying the choice of tools such as Python, Django, and PostgreSQL.

Computer Science Concepts of the Project:

Distributed Systems

A distributed system is a set of independent software and hardware components connected through a network, cooperating to achieve a common objective. Each component operates autonomously while communicating with the others.

In such systems, concepts like availability, fault tolerance, scalability, and load balancing are essential. The DEARS project adopts this approach in order to separate responsibilities between the user interface, the dispatcher, and the emergency response services.

RPC Communication

RPC (Remote Procedure Call) communication allows a program to execute a function located on a remote machine as if it were a local call. This abstraction simplifies data exchange and interaction between distributed services.

In this project, RPC is used to allow the dispatcher to transmit alerts to emergency services (police, firefighters, medical teams), ensuring fast and structured communication between backend components.

REST API

A REST API (Representational State Transfer) is an architectural style based on the HTTP protocol. It uses standard methods such as GET, POST, PUT, and DELETE to exchange data, most commonly in JSON format.

In DEARS, REST APIs are used for communication between the web interface and the dispatcher, enabling citizens to submit emergency alerts and receive responses in a simple and efficient manner.

Relational Databases

Relational databases store data in the form of tables linked together through primary and foreign keys. They ensure data integrity, consistency, and persistence.

Technologies Used:



Figure 1: python logo

Python is an interpreted programming language known for its simplicity, readability, and extensive ecosystem of libraries. It is particularly well-suited for backend development and the implementation of distributed applications. In this project, Python is used to develop the backend services, the dispatcher component, and the RPC communication mechanisms.



Figure 2:django logo

Django is a high-level Python web framework that enables fast development of secure and maintainable web applications. It natively integrates database management, form handling, and API creation. In this project, Django is used to develop the web interface that allows users to submit emergency alerts.



Figure 4:PostgreSQL

PostgreSQL is an open-source relational database management system, well-known for its robustness, performance, and strong compliance with SQL standards. In this project, it is used as the central database to store all critical data and ensure its consistency within a distributed environment.



Figure 5:GitHub logo

Git is a distributed version control system that allows tracking changes in source code and facilitates collaborative work among developers. GitHub is a platform that hosts Git repositories and provides additional project management features. In this project, GitHub is used for source code sharing, branch management, and progress tracking throughout the development process.

RPyC (Remote Python Call) In the DEARS project, we have chosen RPyC to implement the RPC mechanisms. RPyC is a transparent, symmetric library for remote computing in Python that allows services to call functions and access objects over a network as if they were local.

Unlike other RPC frameworks, RPyC provides a "Pythonic" approach that eliminates the need for complex interface definition languages. In our architecture, RPyC is used to allow the Dispatcher to transmit alerts to emergency services (police, firefighters, medical teams) efficiently. This ensures fast, reliable, and structured communication between the backend components while maintaining the high performance required for emergency response.

Conclusion:

This theoretical overview highlights the synergy between the chosen technologies to ensure a robust and scalable architecture. The combination of Django's flexibility, PostgreSQL's power, and the efficiency of RPC forms the technical backbone upon which the detailed analysis of the system requirements is built.

Chapter 3: Requirements Analysis

Introduction:

Before moving into the implementation phase, a rigorous analysis of requirements is essential to translate user expectations into technical functionalities. This chapter details the functional analysis of the system and utilizes UML modeling to structure the interactions between citizens and emergency services.

Functional Analysis

The system allows citizens to report an emergency, while emergency services receive and process these alerts.

UML Modeling

Several UML diagrams were used:

Use Case Diagram:



Figure 6:use case diagram

The Use Case diagram illustrates the main interactions between the DEARS system and its primary actors: the Citizen, the Responder, and the Administrator.

- **Citizen:**
 - A citizen can **register an account** and **submit an emergency alert** to the system.
 - Once an alert is submitted, the citizen can **view the alert status** and **review their alerts** at any time.
 - These actions interact with the system dispatcher, which then routes the alert to the appropriate emergency service.
- **Responder:**
 - Responders (Police, Firefighters, Medical teams) can **view their assigned missions**, which are dispatched alerts related to their service.
 - They can also **update the mission status**, reflecting the progress of the intervention.
- **Administrator:**
 - The administrator is responsible for **managing the response units**, ensuring that every emergency service has the necessary active units.
 - They can **monitor all alerts** through the dashboard and **update alert statuses** if needed.
 - The dashboard functionality extends monitoring, providing a global operational view of the system.
- **Dispatcher (System Internal Role):**
 - The diagram also highlights an internal process: the dispatcher **routes the alert to the appropriate service** using a distributed communication mechanism.
 - This action is triggered by the citizen's alert submission and ensures proper coordination in the system.

Class Diagram:

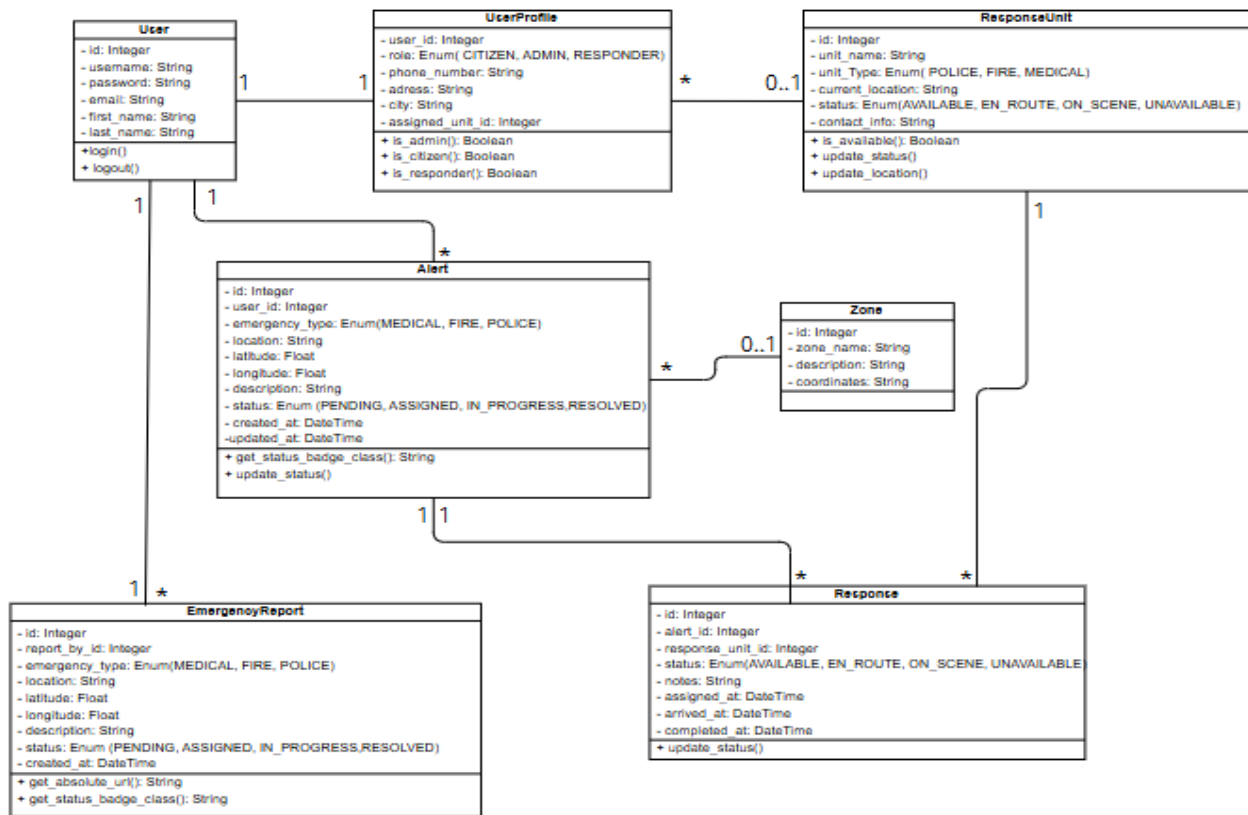


Figure 7:Class Diagram

The Class Diagram illustrates the structural design of the DEARS system by showing its main entities, their attributes, behaviors, and the relationships between them. It defines how data is stored, managed, and used across the distributed components of the system.

Main Entities

- **User**

Represents individuals interacting with the system. Each user has authentication attributes such as username, password, and email. A user can log in and log out of the system.

- **UserProfile**

Extends the User entity by defining the role within the system: *Citizen*, *Admin*, or *Responder*. It also includes additional details like phone number and assigned response unit if the user is a responder. There is a **one-to-one** relationship between User and UserProfile.

- **Alert**

Stores all emergency alerts submitted by citizens. Each alert includes a location, description, coordinates, emergency type (Medical, Fire, Police), and a status (New, Dispatched, In Progress, Resolved). An alert is associated with exactly one citizen and may be assigned to a specific intervention unit.

- **ResponseUnit**

Represents the emergency units (police cars, fire trucks, ambulances). Each has a unit type, location, and operational status (Available, En Route, On Site, Completed). A response unit can be linked to multiple alerts over time.

- **Zone**

Defines geographical areas where alerts may occur. A zone can contain multiple alerts and multiple response units, establishing contextual organization of intervention activities.

- **Response**

Manages the lifecycle of an alert once assigned to a response unit. It stores timestamps for assignment and completion, along with the current response status.

- **EmergencyReport**

Provides a historical record of emergency actions taken. It logs every interaction related to a submitted alert, including reporter information, timestamps, and processing status. This allows traceability and reporting for administrative and legal purposes.

Sequence Diagram:

Citizen :

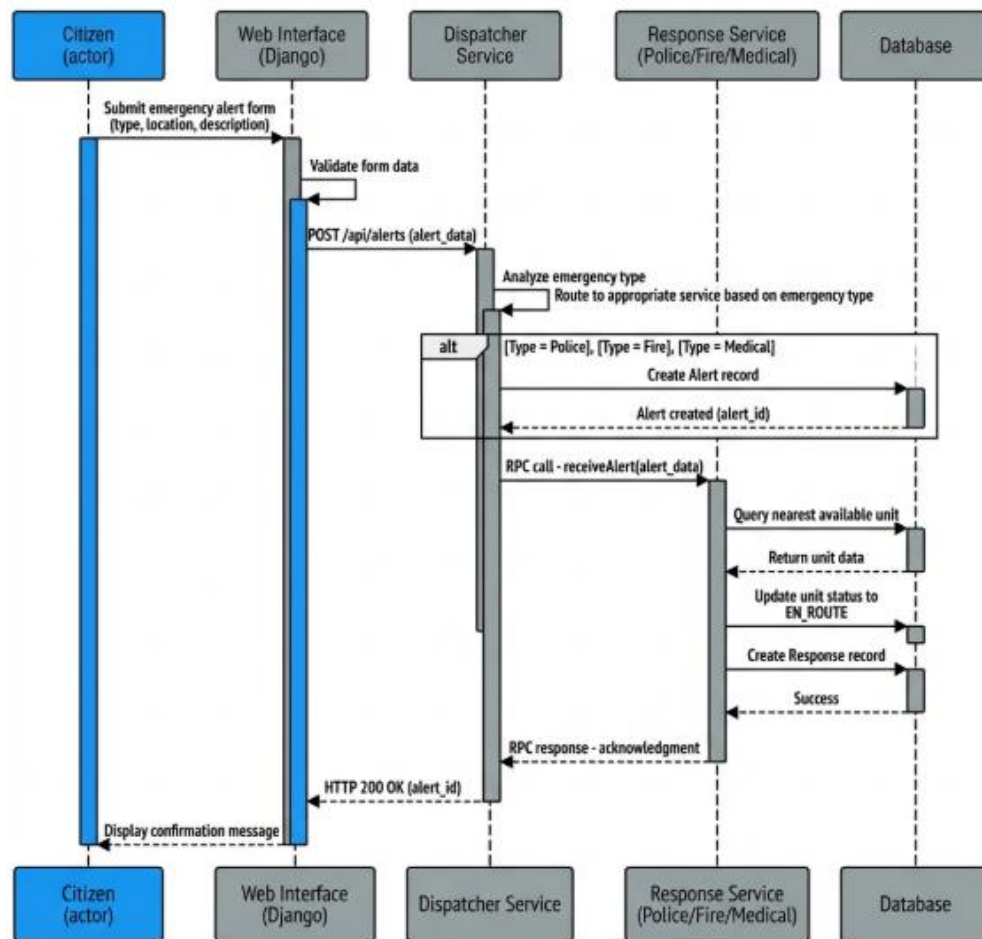


Figure 8:sequence diagram (citizen)

This sequence diagram illustrates the step-by-step workflow of submitting and processing an emergency alert within the DEARS system.

1. **The process begins with the Citizen**, who fills out and submits an emergency alert form through the **Web Interface (Django)**.
The form contains information such as emergency type, location, and description.
2. The **Web Interface validates the submitted data** and sends an HTTP POST request to the **Dispatcher Service** through the REST endpoint (POST /api/alerts).

3. Upon receiving the alert data, the **Dispatcher analyzes the type of emergency**. Based on the alert category (Police, Fire, Medical), the dispatcher routes the request to the corresponding **Response Service** using a Remote Procedure Call (RPC).
4. The **Dispatcher Service creates a new Alert record** in the database and sends an RPC command (receiveAlert(alert_data)) to the selected Response Service.
5. The **Response Service queries the database** to find the nearest available response unit to the alert location.
6. Once the closest unit is identified:
 - The unit status is updated to **EN_ROUTE**
 - A **Response record** is created to track the mission progress
7. The Response Service then sends an **RPC acknowledgement** back to the Dispatcher Service to confirm successful processing.
8. The Dispatcher returns an **HTTP 200 OK response** to the Web Interface.
9. Finally, the **Web Interface displays a confirmation message** to the citizen, indicating that the alert has been successfully submitted and routed.

Administrator Dashboard Interactions

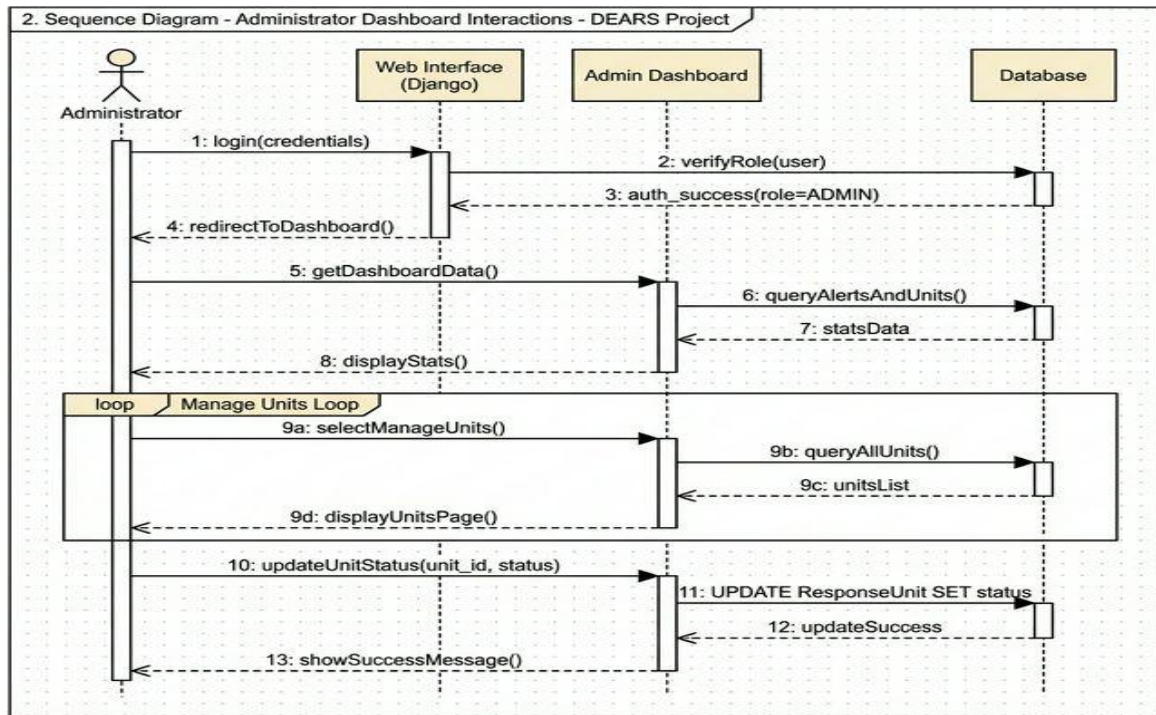


Figure 9:sequence diagram (Admin)

This sequence diagram describes the interaction flow between the Administrator and the system dashboard in the DEARS platform.

1. The **Administrator logs into the system** by submitting their credentials via the **Web Interface (Django)**.
2. The system verifies the credentials and **checks the user role**.
3. If the user is recognized as an **admin**, authentication succeeds, and authorization is granted.
4. The Admin is then **redirected to the dashboard**.
5. The Admin Dashboard requests dashboard data, including system statistics.
6. A database query is performed to retrieve **alerts information and response unit data**.
7. The database returns **aggregated statistics**, which are sent back to the Admin Dashboard.
8. The dashboard **displays statistics** such as the number of alerts and active intervention units.

Management of Intervention Units (Loop Section)

9. The Admin selects the **Manage Units** feature from the dashboard menu.
10. A request is issued to query the database for the list of **response units**.
11. The database returns the complete list of registered units.
12. The **Manage Units** page is displayed to the administrator with response unit details.
13. When the admin updates the **status of a response unit**, an update request is sent to the database.
14. The database successfully updates the record and returns confirmation.
15. The dashboard then **displays a success message** to confirm the update.

Responder:

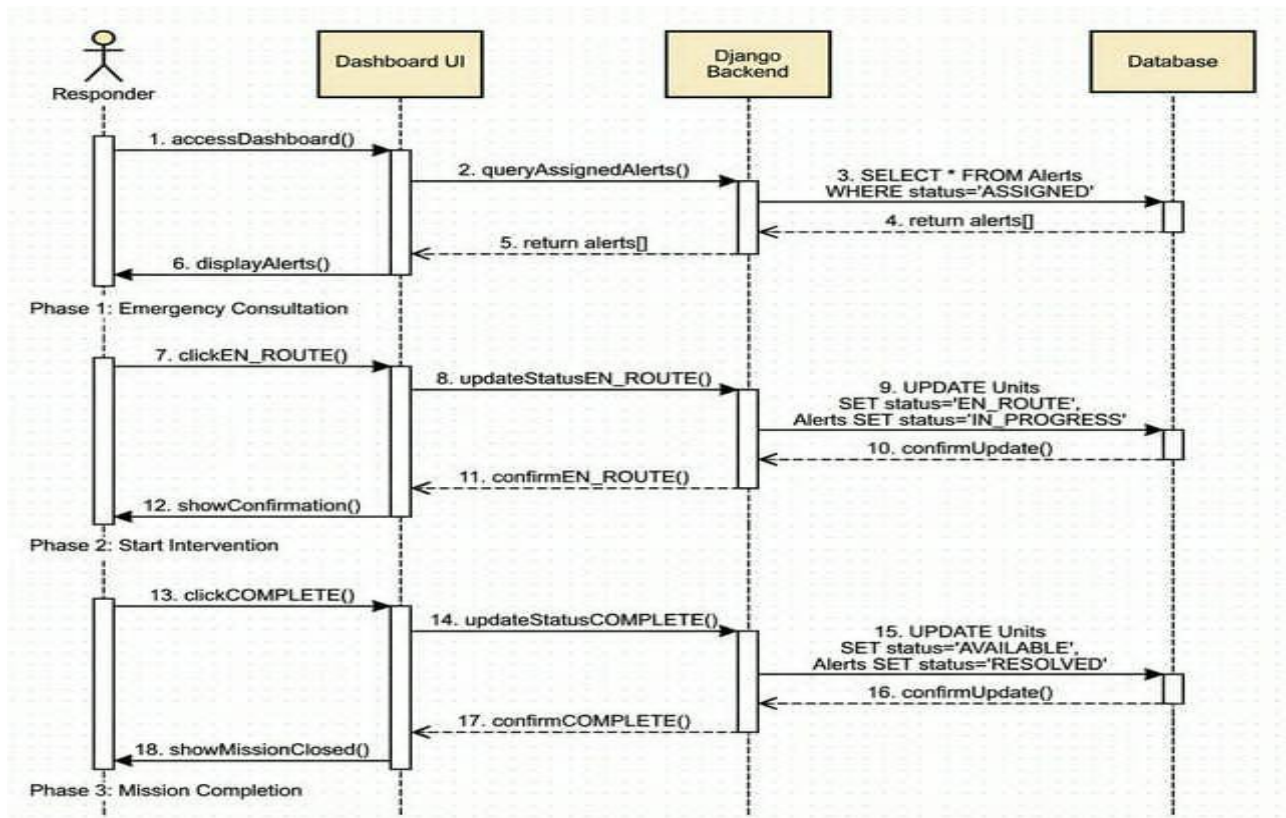


Figure 10:sequence diagram (responder)

Phase 1: Emergency Consultation

1. The **Responder accesses their dashboard** to check missions assigned to them.
2. The Dashboard UI queries the backend for alerts assigned to that responder.
3. The **backend retrieves alerts from the database** where alert status = “ASSIGNED”.
4. The database returns the list of alerts.
5. The backend sends the data to the UI.
6. The **dashboard displays the assigned alerts**.

When the responder selects a mission:

7. The responder clicks on “**EN_ROUTE**” to indicate that they are on their way.
8. The Dashboard UI sends a request to update alert and unit statuses.
9. The database updates:
 - The unit status → **EN_ROUTE**
 - The alert status → **IN_PROGRESS**
10. Confirmation is returned to the backend.
11. Backend confirms the update to the UI.
12. The UI displays a **confirmation message** to the responder.

Phase 2: Start Intervention

13. When the responder arrives on-site, they click “**COMPLETE**”.
14. Another update request is issued to the backend.
15. The database updates statuses again:
 - The unit status → **AVAILABLE**
 - The alert status → **RESOLVED**
16. The database confirms the update to the backend.
17. The backend confirms success to the UI.
18. The **dashboard shows a mission completion message**.

Deployment Diagram:

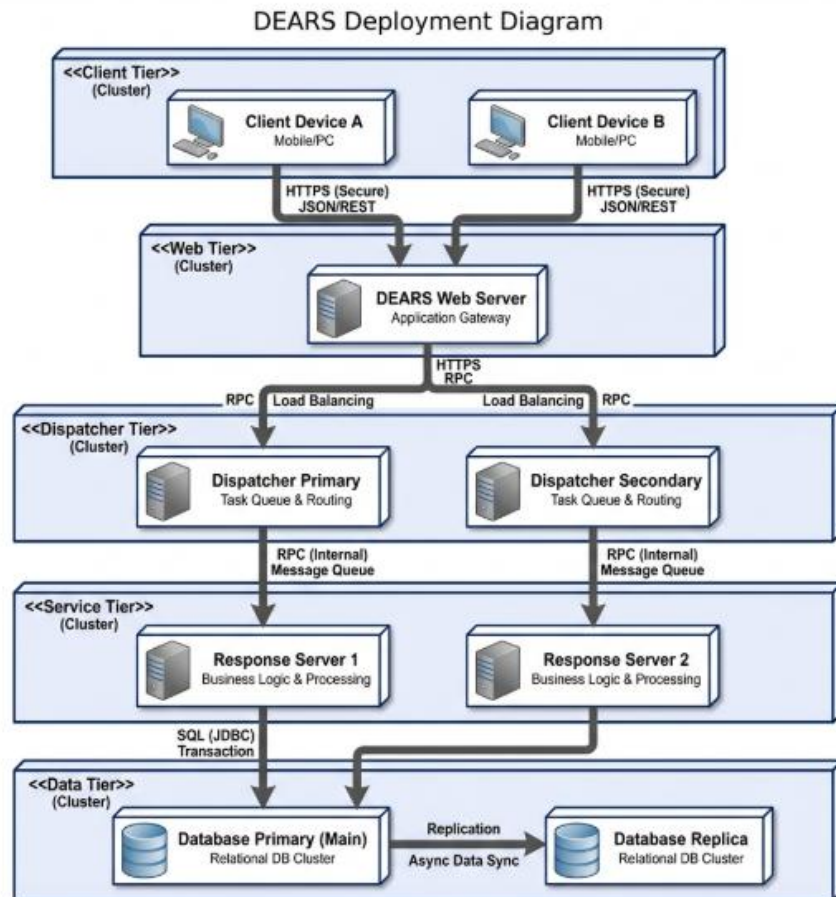


Figure 11 Deployment Diagram

1. Client Tier (Cluster)

Devices: This layer represents the end-users, such as citizens using mobile devices or PCs to submit alerts.

Communication: These devices communicate with the system using HTTP, JSON/REST protocols, ensuring a lightweight and standard way to transmit data.

2. Web Tier (Cluster)

DEARS Web Server: This serves as the Application Gateway.

Role: It acts as the entry point for the system, handling incoming REST API requests from the web interface.

Integration: It bridges the gap between the public-facing internet and the internal backend services using RPC.

3. Dispatcher Tier (Cluster)

Components: Includes Primary and Secondary Dispatchers to ensure fault tolerance.

Mechanism: It uses Load Balancing to distribute incoming alerts and manages Task Queues & Routing.

Function: The dispatcher is responsible for the automatic routing of alerts to the appropriate emergency service units.

4. Service Tier (Cluster)

Response Servers: These servers handle the core Business Logic & Processing.

Communication: They receive routed messages from the Dispatcher Tier via Internal RPC/Message Queues.

Management: This layer manages intervention units and processes the specific requirements of the emergency alert.

5. Data Tier (Cluster)

Database Primary (Main): A relational database (PostgreSQL) that handles SQL (JDBC) Transactions to ensure data integrity and consistency.

Database Replica: To ensure reliability and fault tolerance, the system uses Async Data Sync (Replication) to a secondary database cluster.

Persistence: This tier is responsible for the permanent storage and tracking of all alerts submitted through the system.

Conclusion:

The analysis conducted in this chapter has clarified the role of each actor and finalized the system specifications. This structured vision, illustrated by use case and sequence diagrams, serves as the roadmap for the software implementation described in the practical section.

Part II: Practical Study

Chapter 1: Application Development

Introduction:

The development phase represents the core of this project, where theoretical concepts are translated into functional code. This chapter details the development environment, the implementation of business logic within the backend, and the creation of an intuitive user interface designed for rapid alert submission.

Development Environment

- Visual Studio Code
- Python & Django
- PostgreSQL (Neon Cloud Database)

Backend Implementation

The backend services handle the business logic and the RPC-based communication between the distributed components of the system.

Frontend Implementation

The frontend allows users to submit emergency alerts through a web interface designed for simplicity and efficiency.

Database

The main tasks performed include:

- Creation of the PostgreSQL database
- Configuration of the connection with the project services
- Creation of tables corresponding to the system entities
- Implementation of integrity constraints (primary keys, foreign keys, enumerated types)

This database ensures persistence, consistency, and secure management of data exchanged between the different services.

Conclusion:

The implementation of DEARS demonstrates the viability of a distributed architecture for emergency management. The clear separation between the frontend, the dispatcher, and the backend services ensures not only easier maintenance but also increased system resilience during unexpected traffic surges.

Chapter 5: Testing and Validation

Introduction:

An emergency alert management system cannot be deployed without a rigorous validation phase. This chapter is dedicated to presenting the tests performed on the interface and the key functionalities of the project. We will explore the various application screens and verify their compliance with the requirements defined during the analysis phase.

Home page:

This page represents the home interface of the DEARS system, designed to introduce the platform and its main features. It allows users to access emergency services, create an account, log in, and learn how the distributed alert system operates.

DEARS

Distributed Emergency Alert & Response System

A cutting-edge distributed system that connects citizens with emergency services through intelligent routing and real-time coordination.

[Create Account to Report Emergency](#)
[Login](#)

24/7

Available

<60s

Response Time

3

Service Types

Emergency Services We Coordinate

Comprehensive coverage for all types of emergencies



Police Services

- ✓ Criminal activity reporting
- ✓ Theft and burglary
- ✓ Domestic disturbances
- ✓ Traffic accidents
- ✓ Public safety threats



Fire Services

- ✓ Building fires
- ✓ Vehicle fires
- ✓ Wildfire threats
- ✓ Gas leaks
- ✓ Hazardous materials



Medical Services

- ✓ Medical emergencies
- ✓ Cardiac arrest
- ✓ Severe injuries
- ✓ Respiratory distress
- ✓ Unconscious persons

Advanced System Features

Built with resilience, scalability, and speed in mind



Instant Response

Automated routing to the nearest available emergency unit within seconds. Our intelligent dispatcher ensures zero delay in critical situations.



Distributed Architecture

Resilient system design ensures 99.9% uptime even during high load. Multiple redundant servers guarantee service availability.



Multi-Service Coordination

Integrated Police, Fire, and Medical emergency response coordination. Single platform for all emergency types.



GPS Location Tracking

Precise location detection using GPS coordinates. Helps emergency units reach you faster with accurate positioning.



Secure & Private

End-to-end encryption for all communications. Your personal information and emergency details are protected.



Real-Time Updates

Track your emergency alert status in real-time. Receive notifications when response units are dispatched.

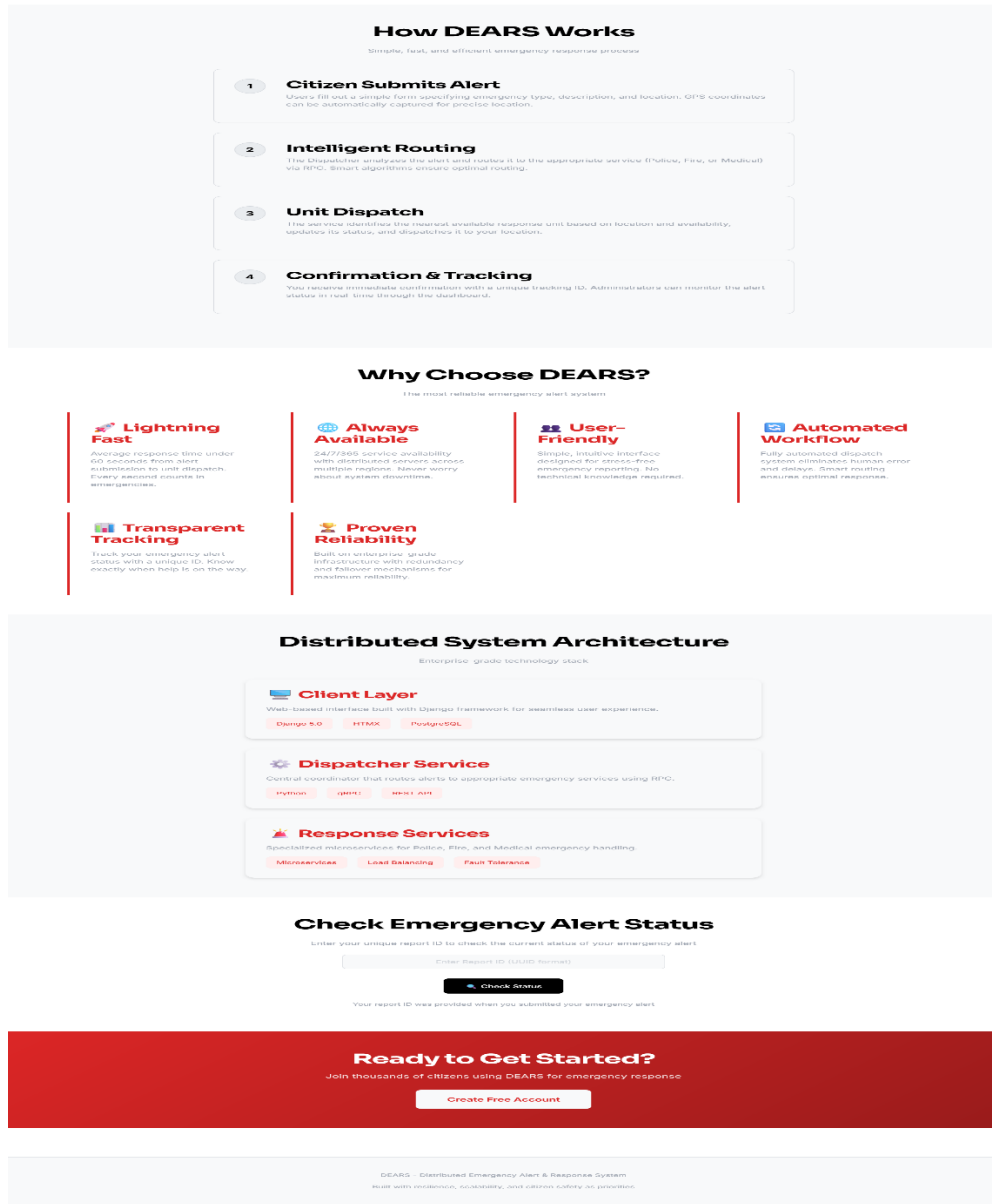


Figure 12:home page

Register page:

This page allows new users to create an account by entering their personal information in order to access and use the DEARS emergency alert services

 **DEARS**

LoginSign Up

Create Account

Join DEARS to report emergencies

Username

Choose a username

First name

Last name

First name

Last name

Email

Email address

Phone number

Phone number

Address

Home address

City

City

Password

Password

Your password must contain at least 8 characters.

Password confirmation

Confirm password

Create Account

Already have an account? [Sign in](#)

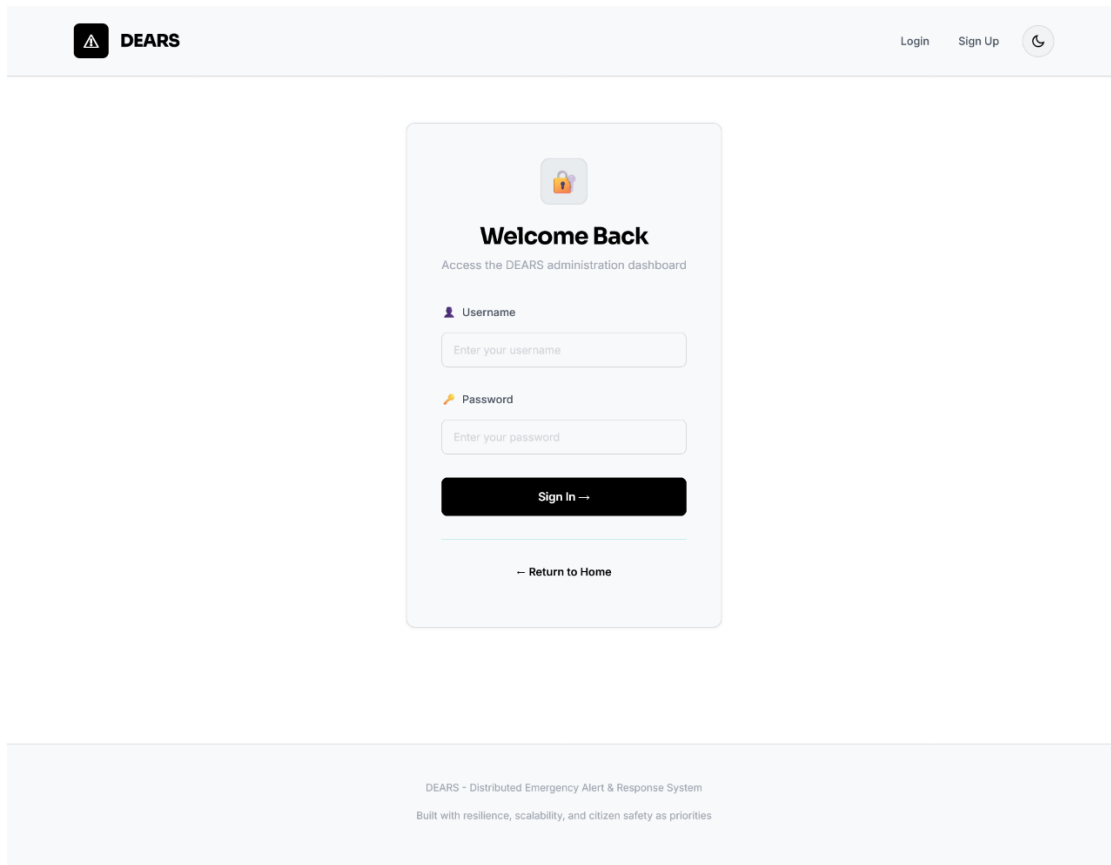
DEARS - Distributed Emergency Alert & Response System

Built with resilience, scalability, and citizen safety as priorities

Figure 13:register page

Login page:

This page allows registered users to sign in by entering their credentials in order to access the DEARS platform and its features.



The screenshot displays the DEARS login interface. At the top, a header bar contains the DEARS logo on the left and 'Login', 'Sign Up', and a circular refresh icon on the right. The main content area features a central card with a lock icon, the heading 'Welcome Back', and the subtitle 'Access the DEARS administration dashboard'. Below this are two input fields: 'Username' with the placeholder 'Enter your username' and 'Password' with the placeholder 'Enter your password'. A black 'Sign In →' button is positioned below the password field. A horizontal line separates this from a '← Return to Home' link. The footer of the page contains the text 'DEARS - Distributed Emergency Alert & Response System' and 'Built with resilience, scalability, and citizen safety as priorities'.

Figure 14:login page

Declare emergency page :

This page allows the citizen to submit an emergency alert by selecting the type of emergency and providing detailed information about the situation and their location.

 **DEARS**

Report EmergencyMy AlertsAnass AllabouchLogout



DEARS

Distributed Emergency Alert & Response System

Submit your emergency alert and help will be dispatched immediately

Emergency Type *

**Police**
Criminal activity, theft, or
urgent police assistance

**Fire**
Fire, smoke, or fire-related
emergencies

**Medical**
Medical emergencies, injuries,
or health crises

Your Information

Name:
Anass Allabouch

Phone:
0600000000

Address:
address, Agadir

Emergency Description *

HELLO!!

Provide clear details about the situation, location landmarks, and any immediate dangers

Location Description *

Hay Salam

Latitude *

30.397106

-9.548337

Accurate location helps emergency services reach you faster

Use Current Location

Submit Emergency Alert

Important Information

- This system dispatches real emergency services. Only use for genuine emergencies.
- Your alert will be automatically routed to the appropriate service based on emergency type.
- The nearest available response unit will be dispatched to your location.
- Keep your phone accessible for callback from emergency services.

DEARS - Distributed Emergency Alert & Response System

Built with resilience, scalability, and citizen safety as priorities

Figure 15:declare emergency page

Status page :

This page confirms that the emergency alert has been successfully submitted and provides real-time tracking details, along with safety instructions for the citizen while emergency units are being dispatched.

 **DEARS**

[Report Emergency](#) [My Alerts](#) [Anass Allabouch](#) [Logout](#) 

Emergency alert declared successfully! Alert ID: 66

Emergency services have been notified. Help is on the way.



Alert Submitted Successfully!

Real-time tracking for your emergency alert #{{ report.alert_uid }}

CURRENT STATUS	LOCATION	REPORTED AT
ASSIGNED	30.397106, -9.548337	December 29, 2025 - 15:59



Alert Dispatched

Your emergency alert has been dispatched to response services. Units are being assigned.



Important: What To Do While Waiting

 **Stay Safe**

- If safe, remain in your current location
- Move to a secure area away from immediate danger
- Do NOT attempt to confront dangerous situations yourself

 **Keep Communication Open**

- Keep your phone charged and accessible
- Answer calls from unknown numbers (could be emergency services)
- Do NOT hang up if you called 911

 **Police Emergency - Safety First:**

- Find a safe, well-lit area if possible
- Lock yourself in a room if threatened
- Comply immediately with police instructions

[Refresh Status](#)

[My Alerts History](#)

[Return Home](#)

DEARS - Distributed Emergency Alert & Response System

Built with resilience, scalability, and citizen safety as priorities

Figure 16:status page

My alerts page:

This page allows the citizen to view all emergency alerts they have submitted, along with their current status, location, and access to detailed tracking information for each alert.

The screenshot displays the 'My Emergency Alerts' interface. At the top, a header bar includes the DEARS logo, navigation links for 'Report Emergency', 'My Alerts', and user information for 'Anass Allabouch', along with a 'Logout' button and a refresh icon. The main section is titled 'My Emergency Alerts' and contains five alert cards arranged in two rows. Each card represents a submitted alert with the following details:

- Alert 1 (Top Left):** Status: ASSIGNED. Date: December 29, 2025 - 15:59. Location: Hay Salam. Description: HELP!!!
- Alert 2 (Top Middle-Left):** Status: RESOLVED. Date: December 29, 2025 - 15:11. Location: Hay salam. Description: Help!
- Alert 3 (Top Middle-Right):** Status: PENDING. Date: December 29, 2025 - 15:08. Location: Hay salam. Description: Help
- Alert 4 (Top Right):** Status: PENDING. Date: December 29, 2025 - 15:08. Location: Hay salam. Description: Help
- Alert 5 (Bottom Left):** Status: PENDING. Date: December 29, 2025 - 15:00. Location: Hay salam. Description: Help

Each card features a 'View Status & Details' button. The footer of the page states: 'DEARS - Distributed Emergency Alert & Response System' and 'Built with resilience, scalability, and citizen safety as priorities'.

Figure 17:my alerts page

Responder:

Responder page:

This page displays the active emergency alert assigned to the responder, including alert details, real-time map location, and the ability to update the mission status during intervention.

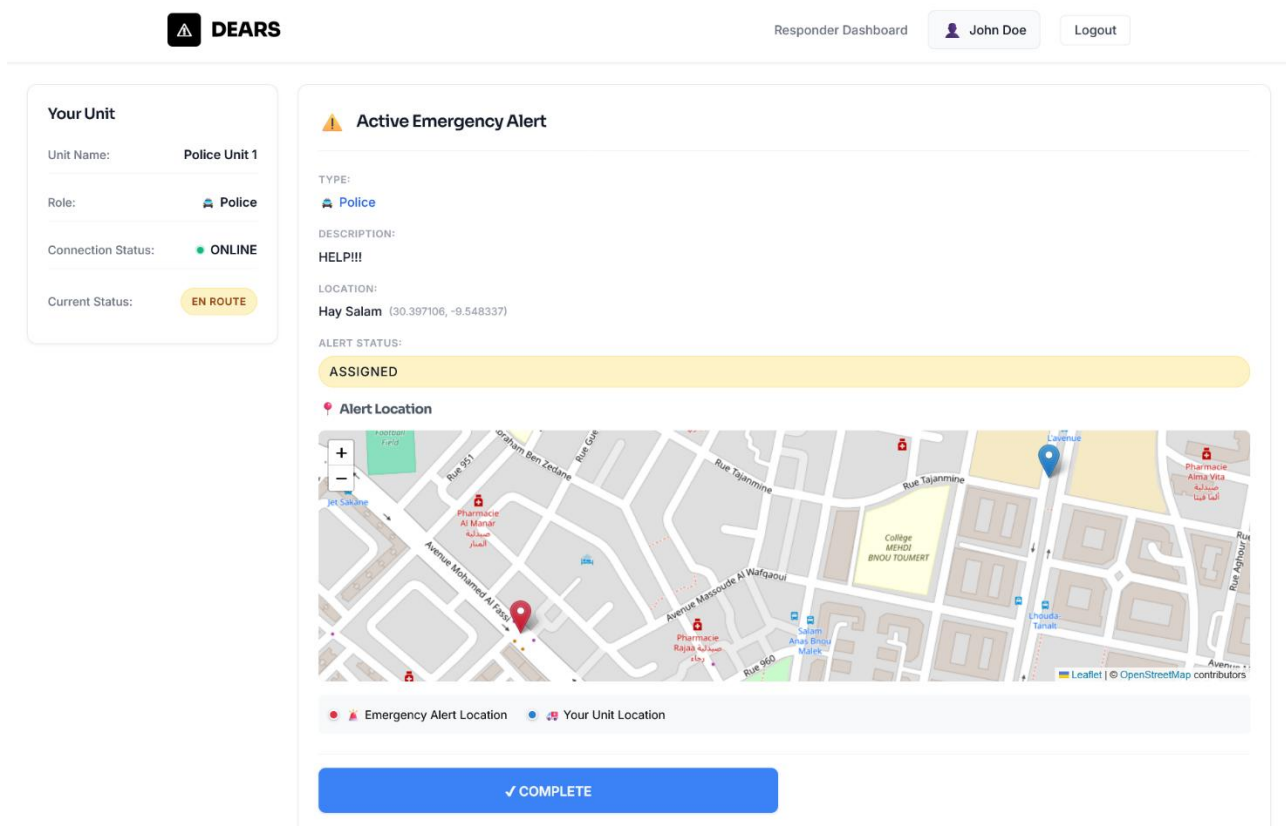
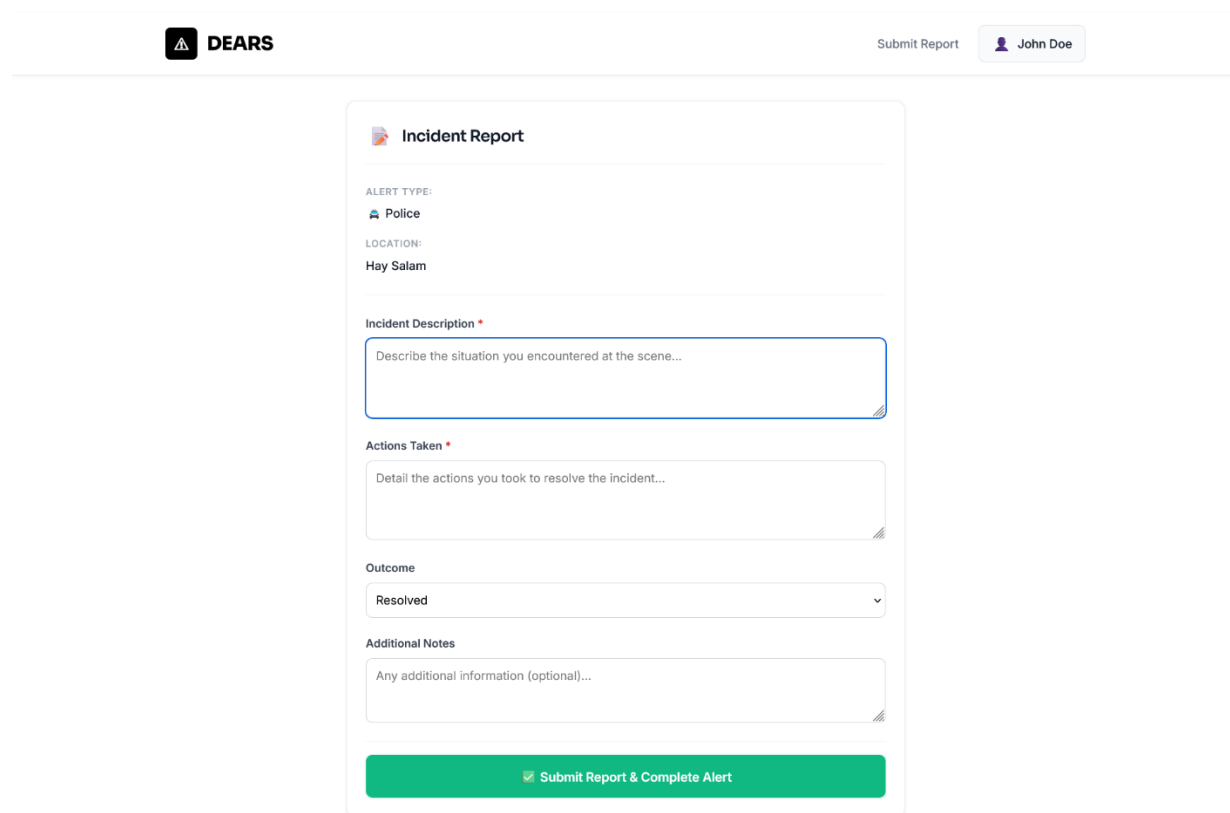


Figure 18: responder page

Submit-report page :

This page allows the responder to submit a detailed incident report, including the situation encountered, actions taken, and the final outcome, in order to officially complete the emergency alert.



The screenshot shows the 'Submit Report' page in the DEARS system. The header includes the DEARS logo, a 'Submit Report' button, and a user profile for 'John Doe'. The main form is titled 'Incident Report' and contains several sections: 'ALERT TYPE:' with a 'Police' selection, 'LOCATION:' with 'Hay Salam', 'Incident Description' with a text area, 'Actions Taken' with a text area, 'Outcome' with a dropdown menu set to 'Resolved', and 'Additional Notes' with a text area. A green button at the bottom is labeled 'Submit Report & Complete Alert'.

DEARS Submit Report John Doe

Incident Report

ALERT TYPE:
Police

LOCATION:
Hay Salam

Incident Description *

Describe the situation you encountered at the scene...

Actions Taken *

Detail the actions you took to resolve the incident...

Outcome
Resolved

Additional Notes
Any additional information (optional)...

Submit Report & Complete Alert

Figure 19:submit-report page

Admin:

Dashboard alerts:

This page provides the administrator with a centralized dashboard to monitor all emergency alerts in the system, view their current status, and perform management actions such as updating or removing alerts.

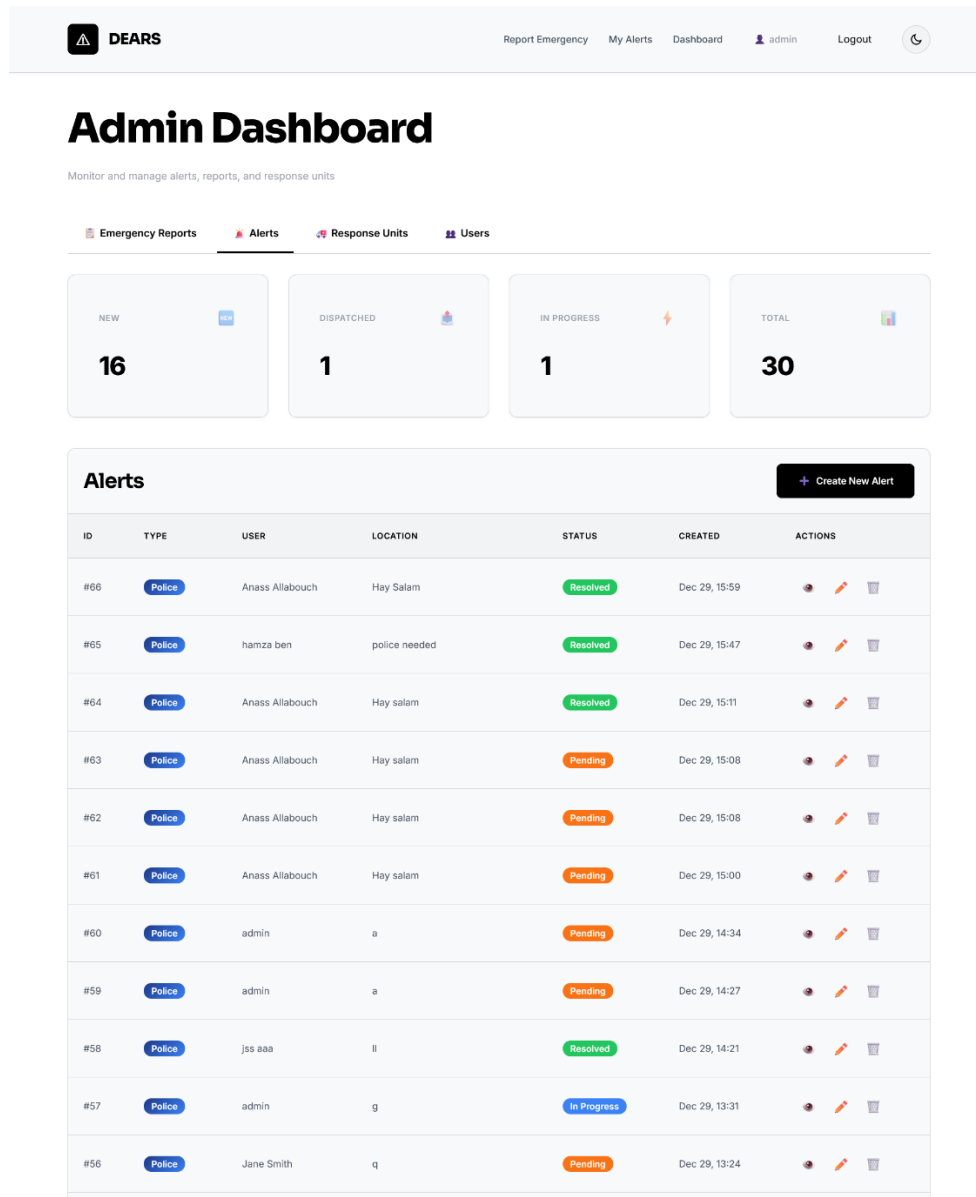


Figure 20:dashboard alerts

Dashboard response page:

This page allows the administrator to manage all emergency response units in the system, including their availability, location, and contact information, with actions to edit or delete each unit.

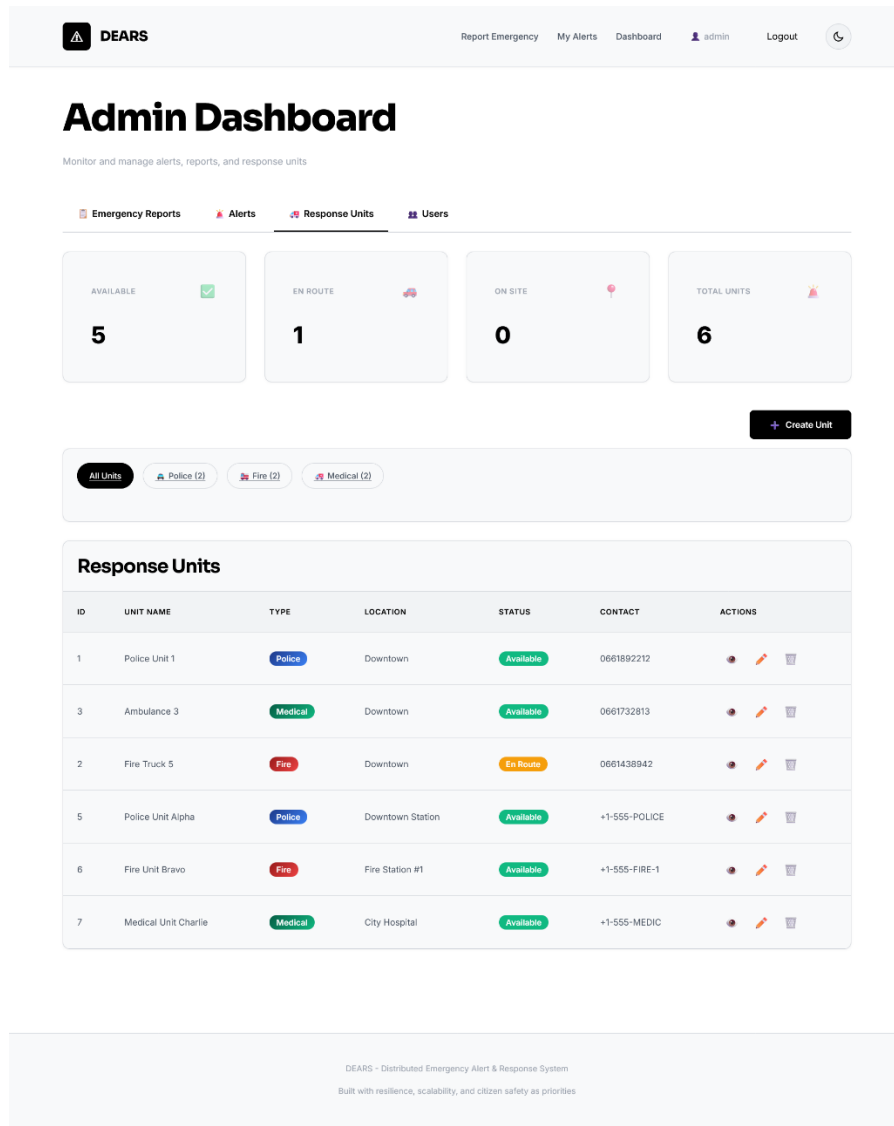


Figure 21:Dashboard response page

Profiles:

Responder's profile page:

This page displays the responder's profile information, including account details and their assigned response unit, along with its current availability and connection status.

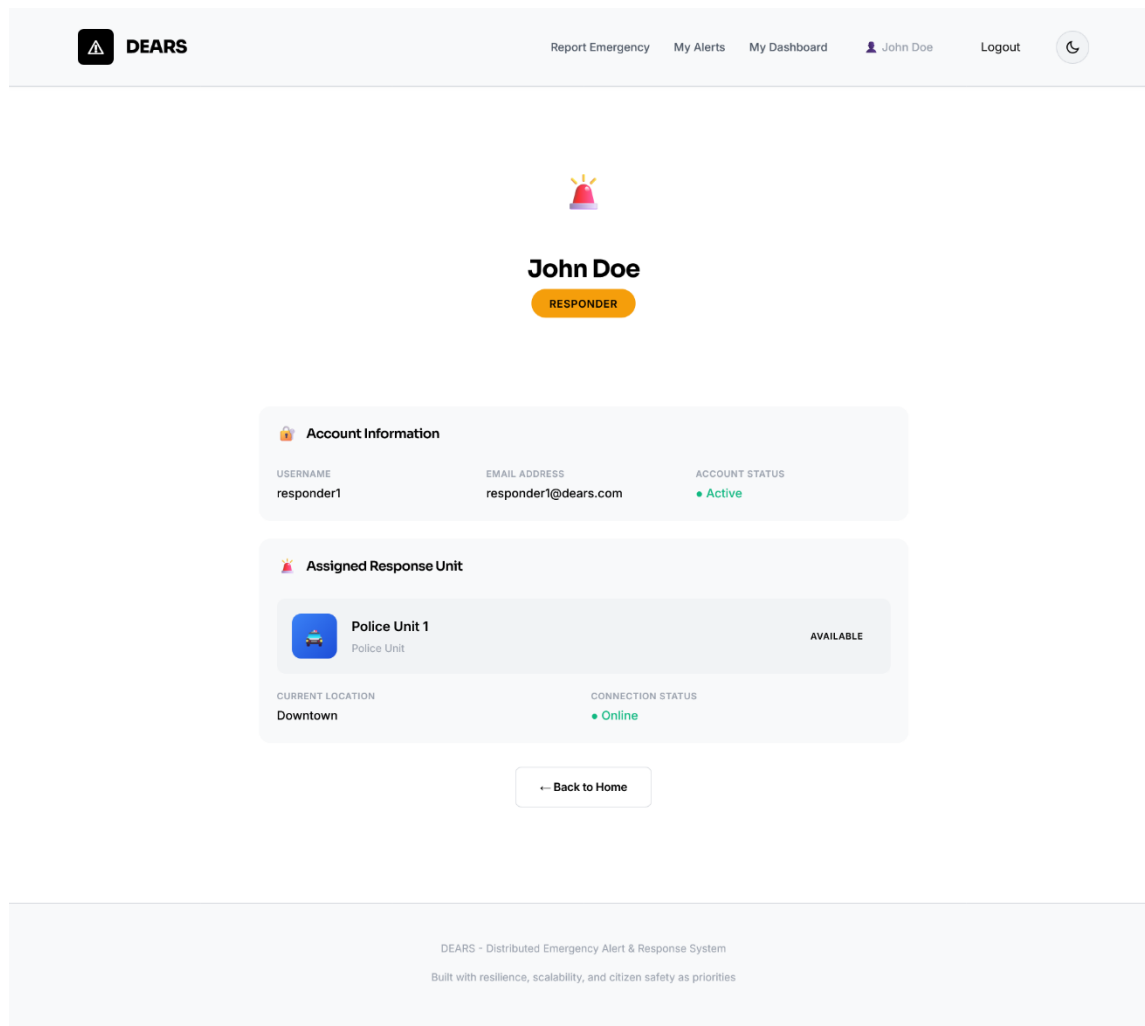


Figure 22: responder's profile page

Admin's profile page :

This page displays the administrator's account information and provides access to administrative tools, including the system management dashboard and the Django administration panel

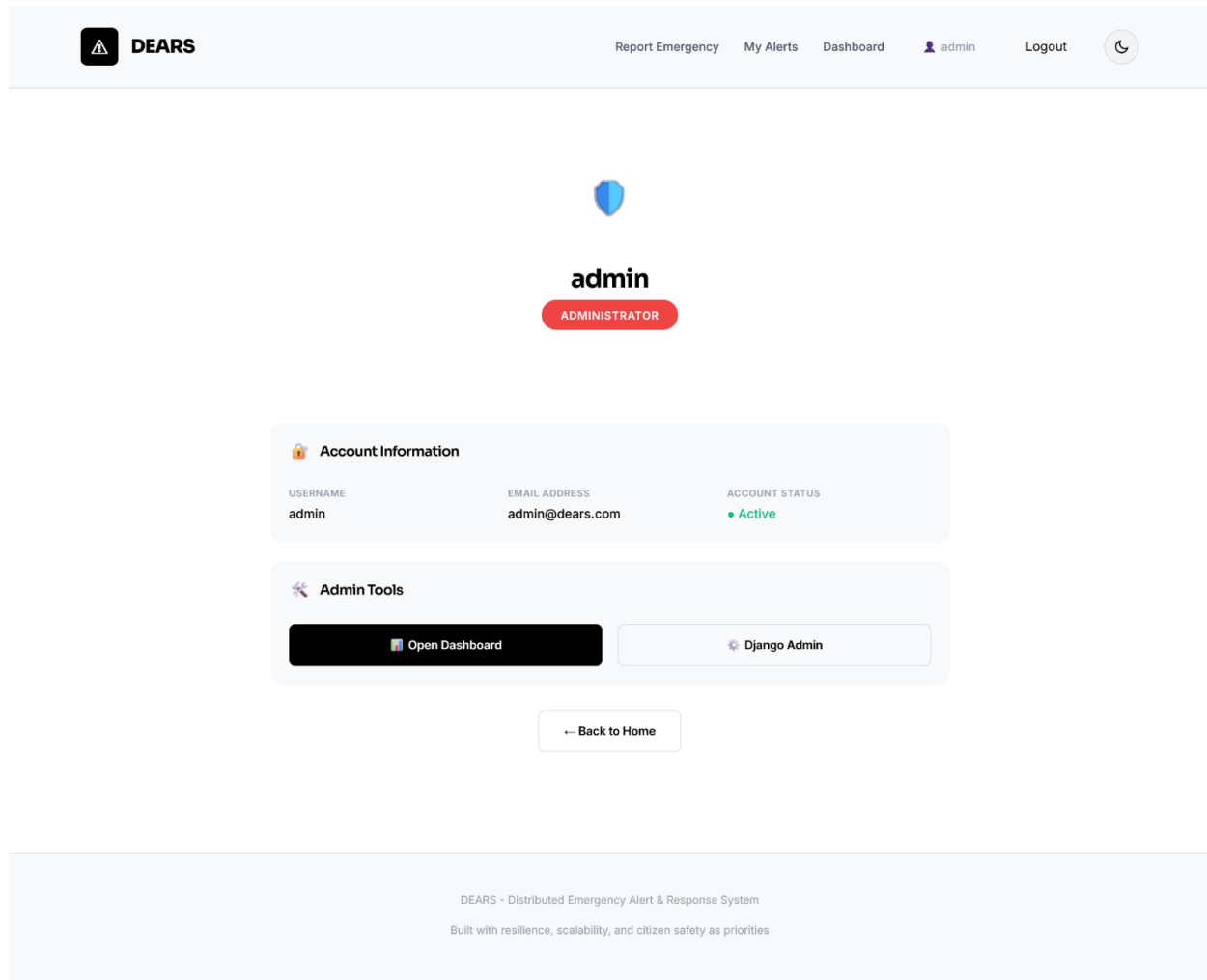


Figure 23:Admin's profile page

citizen's profile page :

This page displays the citizen's personal and account information, allowing them to review their contact details and ensure their profile remains up to date for emergency response.

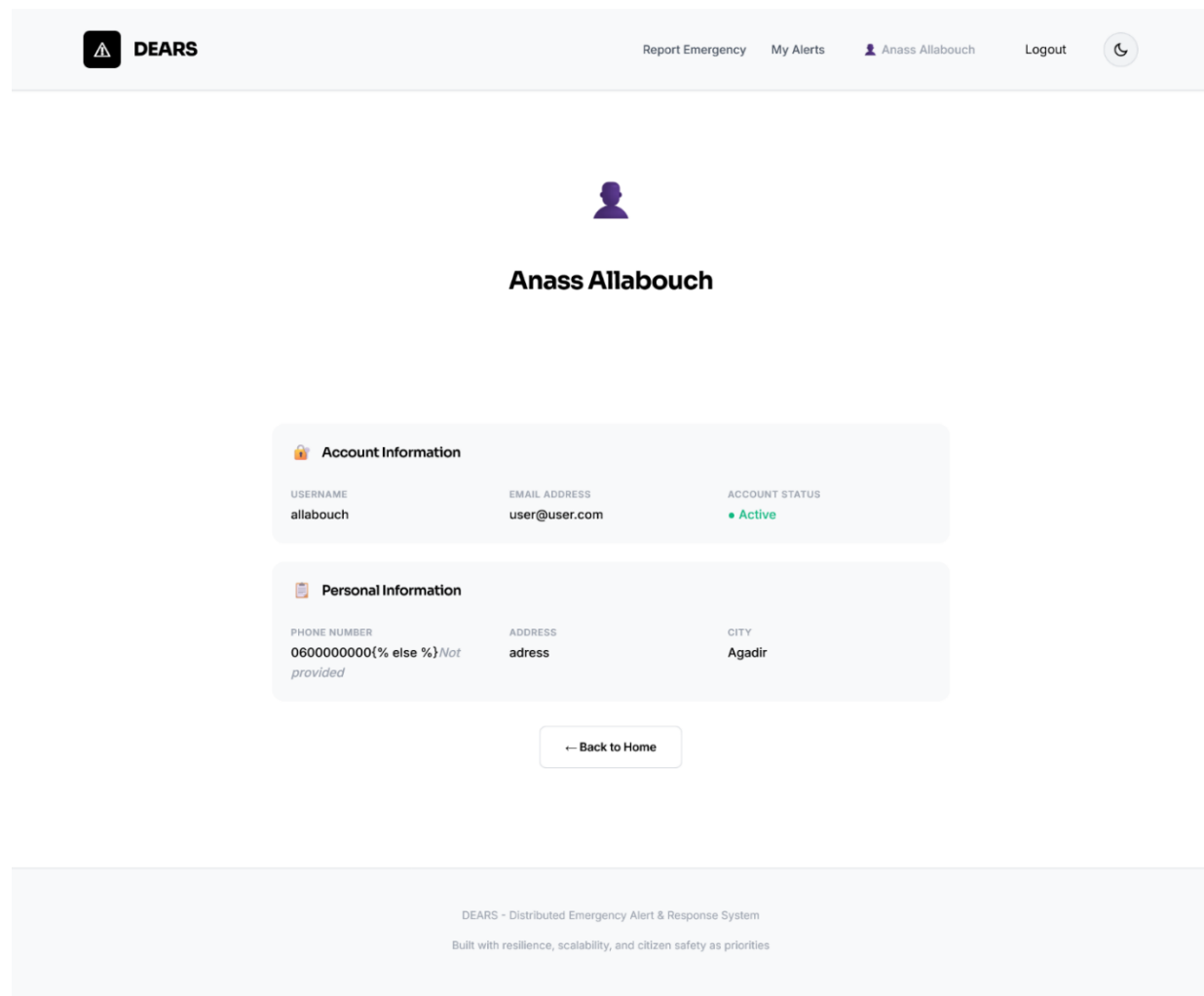


Figure 24: citizen's profile page

Conclusion:

The tests performed confirm that the DEARS system meets its established objectives: alerts are correctly routed, data is securely persisted, and the interface remains accessible. This final step validates the robustness of our distributed solution and opens the door to potential future improvements.

Conclusion:

The development of the Distributed Emergency Alert & Response System (DEARS) enabled us to apply theoretical knowledge in distributed systems to a real-world and critical context. Through the implementation of a multi-service architecture, we ensured that emergency alerts can be efficiently submitted, routed, processed, and tracked in real time.

This project highlighted several key benefits of distributed technologies, including scalability, resilience, and independent service deployment. By integrating RPC-based communication between backend services and a REST interface for user interactions, DEARS guarantees fast coordination and reliable data exchange. The use of PostgreSQL as a central data store further ensured consistency, security, and persistence across the entire system.

Moreover, the project strengthened our experience in software engineering, from UML-based modeling to full-stack development using Python and Django. The implementation of user roles (Citizen, Responder, Administrator) demonstrated a flexible and secure access management approach tailored to emergency operations.

In conclusion, DEARS constitutes a functional and robust emergency response platform capable of improving citizen safety and optimizing the work of emergency responders. The system may evolve in the future through advanced features such as automated geolocation tracking, mobile application support, predictive dispatching based on AI, and enhanced interoperability with real emergency infrastructures.